

LEZIONE 3 – SSD, CONTRATTI, E ARCHITETTURA LOGICA

Questa lezione conclude la parte di analisi del sistema di rubrica telefonica, trattando i diagrammi di sequenza di sistema (SSD) e i contratti, e introduce la parte progettuale definendo l'architettura logica. Di seguito viene riportata la descrizione del sistema Rubrica:

Un utente del sistema – identificato da uno username – può gestire una rubrica telefonica, che include una lista di contatti (massimo 100). Ogni contatto è caratterizzato da un nome univoco, una descrizione, e uno o più numeri di telefono (fisso casa, ufficio, cellulare, etc). L'utente può inserire nuovi contatti nella rubrica telefonica, rimuovere quelli presenti, o aggiornarne i dati. I contatti possono essere aggiunti in modalità "protetta", in modo da impedire la rimozione dalla rubrica. Il sistema deve anche consentire la ricerca di un contatto presente in rubrica, inserendo il nome del contatto oppure un numero di telefono. Il sistema prevede anche un utente amministratore che, oltre alle attività precedenti, può monitorare le attività svolte dagli altri utenti.

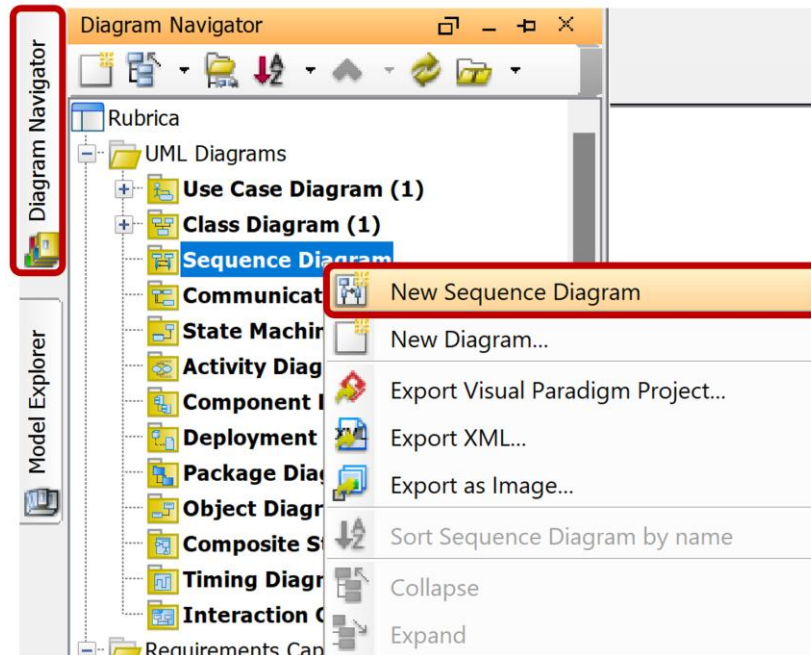
Creazione di un SSD

Per modellare gli SSD utilizzeremo i diagrammi di sequenza UML.

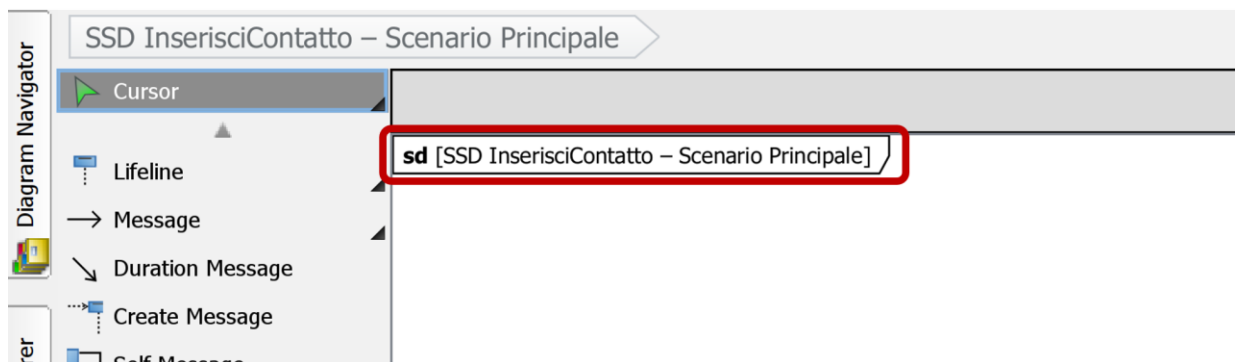
Vogliamo modellare le interazioni descritte dal caso d'uso **InserisciContatto**, di cui si riporta lo scenario principale di successo relativo all'aggiunta di un nuovo contatto in rubrica:

1. Il caso d'uso inizia quando l'Utente apre la propria rubrica
 2. Il sistema mostra l'elenco dei contatti attualmente in rubrica
 3. L'Utente specifica di voler inserire un nuovo contatto in rubrica
 4. Il sistema chiede di inserire i dati del contatto (nome, descrizione e uno o più numeri di telefono)
 5. L'Utente inserisce nome e descrizione
 6. L'Utente inserisce il numero di telefono
- L'Utente ripete il passo 6 finché non conferma di aver terminato l'inserimento
7. Il sistema notifica l'utente dell'inserimento avvenuto correttamente

A tale scopo, a partire dal progetto Rubrica in Visual Paradigm (VP) prodotto nelle lezioni precedenti, dal pannello **Diagram Navigator** a sinistra, clicchiamo con il tasto destro su **Sequence Diagram** e dal menu selezioniamo la voce **New Sequence Diagram**.

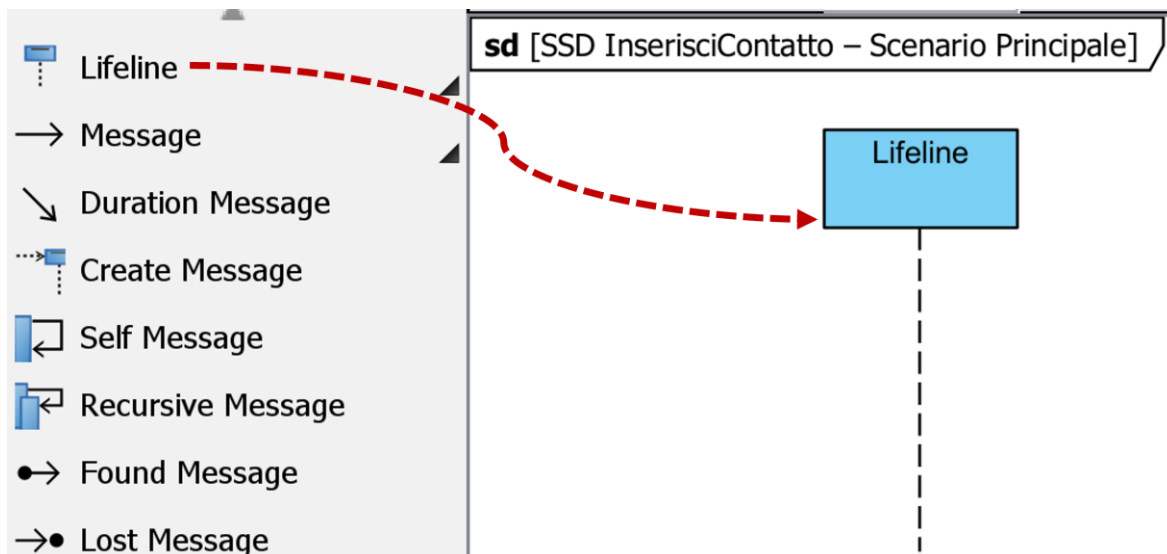


Una volta creato il diagramma di sequenza vuoto, rinominiamolo come già visto in precedenza (doppio click sul nome) e inseriamo “SSD InserisciContatto – Scenario Principale”.

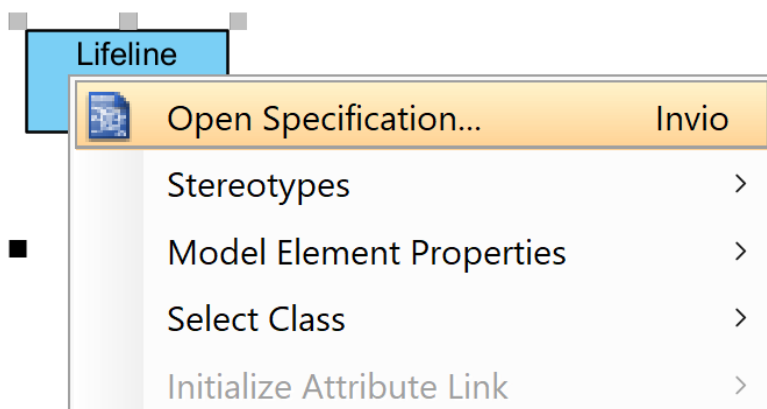


Inserimento delle lifeline dei partecipanti nell'SSD

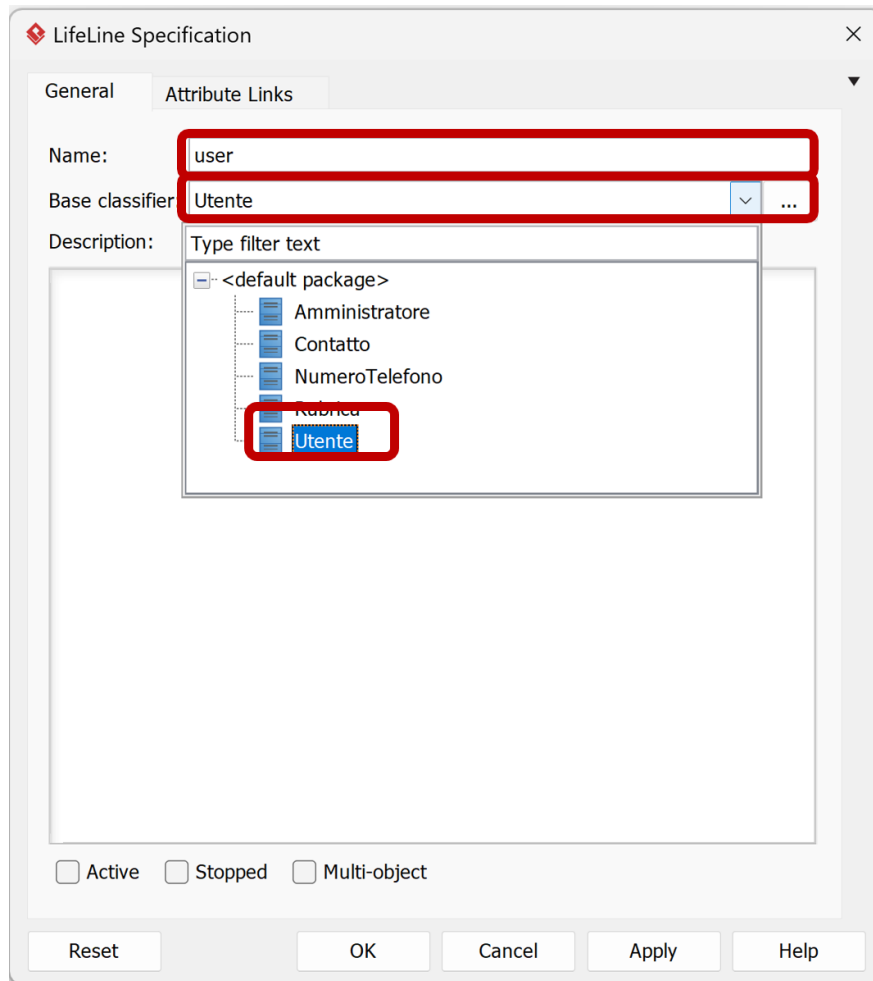
Ora possiamo inserire i partecipanti nell'SSD, che in base al caso d'uso sono l'attore Utente e il Sistema. Per inserire i partecipanti in VP, dalla **Palette** a sinistra selezioniamo la voce **Lifeline** e trasciniamola nel diagramma.



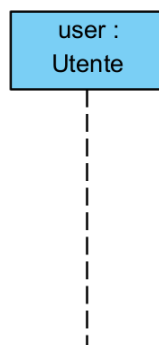
Al momento la lifeline creata non è associata ad alcuna entità del modello. Dato che vogliamo rappresentare l'attore Utente, dobbiamo associare la lifeline a tale concetto. Col tasto destro sulla lifeline selezioniamo la voce **Open Specification**.



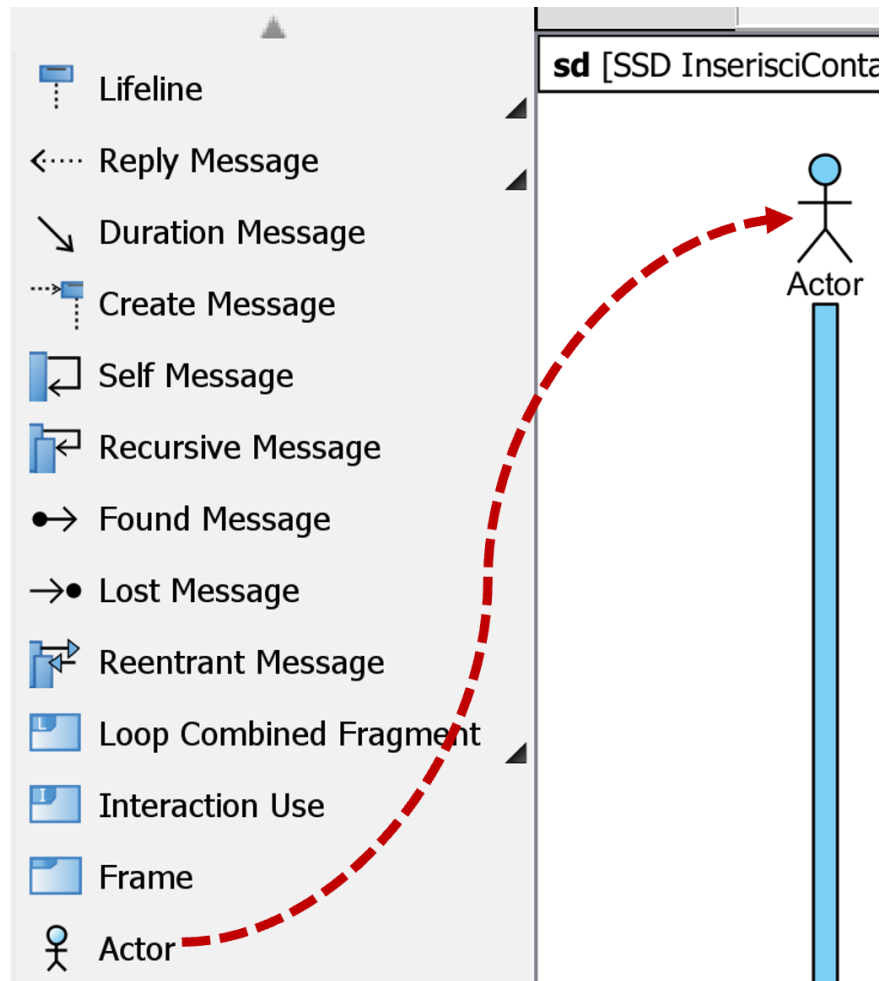
L'operazione aprirà la finestra relativa alle proprietà della lifeline. Nel campo **Name** inseriamo la stringa "user", che rappresenterà un generico utente. Clicchiamo quindi sul menu **Base classifier** ed espandiamo la voce **<default package>**, selezionando la sotto-voce Utente, che dovrebbe essere già presente nel modello in quanto introdotta durante la modellazione dei casi d'uso e del dominio. Infine, clicchiamo su **OK**.



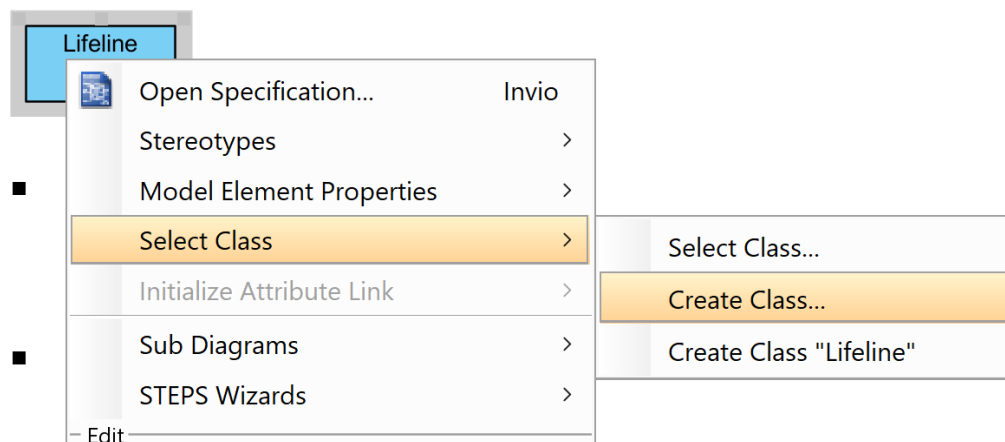
L'operazione produrrà il risultato seguente, associando la lifeline a un Utente chiamato "user".



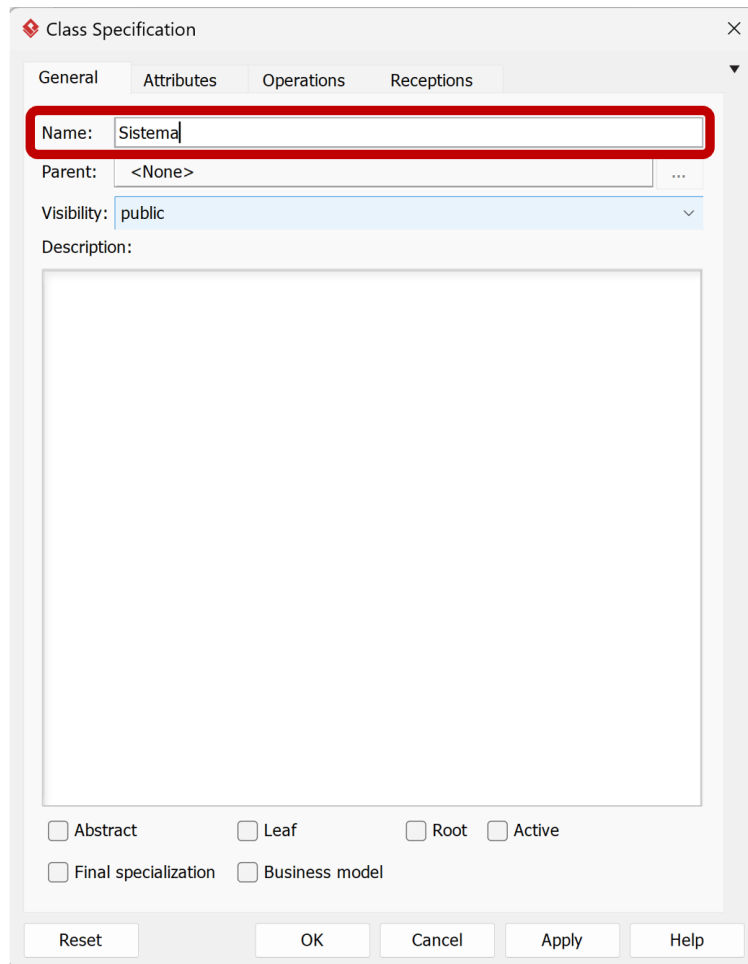
Alternativamente, in sostituzione di una generica lifeline come fatto sopra, nella modellazione di attori negli SSD si può direttamente utilizzare il costrutto UML che rappresenta la figura di attore.



Ripetiamo l'operazione precedente per creare una seconda lifeline relativa al Sistema, in questo caso utilizzando il costrutto classico per le lifeline. Clicchiamo col tasto destro sulla nuova lifeline, e selezioniamo **Select Class > Create Class**. Questa operazione servirà a creare una classe concettuale relativa al sistema Rubrica, da non confondere con la classe Rubrica del modello di dominio, che rappresenta la singola entità di una rubrica telefonica associata a un utente e non l'intero sistema Rubrica che gestisce le rubriche telefoniche (e altre funzionalità) di tutti gli utenti.



Dalla finestra che si aprirà, inseriamo il nome Sistema (così da non confondere il termine con quello di Rubrica) e confermiamo.



Class Specification

General Attributes Operations Receptions

Name: Sistema

Parent: <None>

Visibility: public

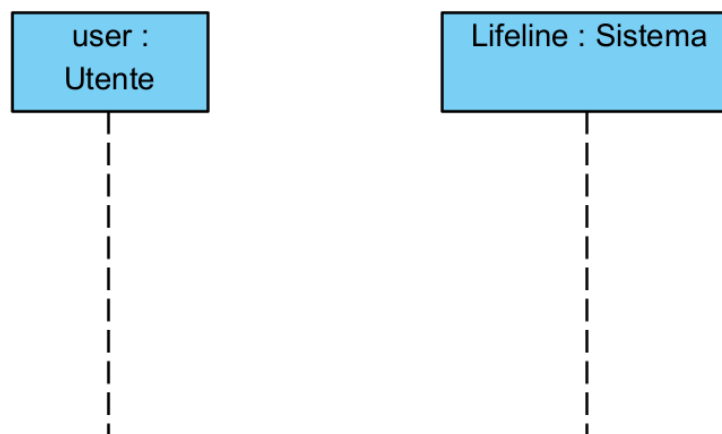
Description:

☐ Abstract ☐ Leaf ☐ Root ☐ Active

☐ Final specialization ☐ Business model

Reset OK Cancel Apply Help

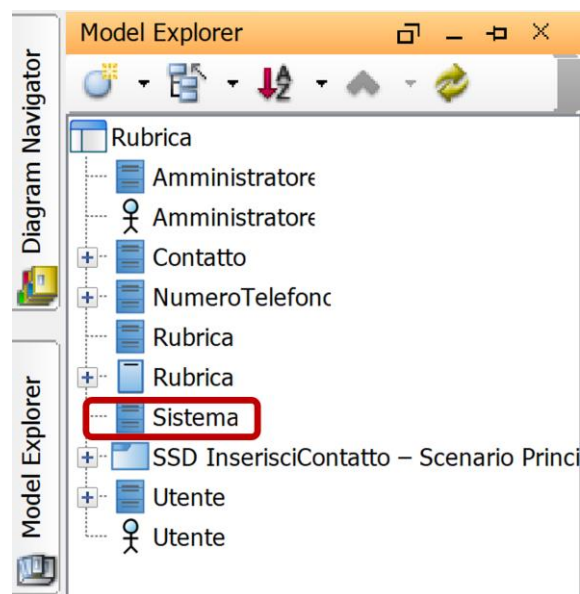
Il risultato sul diagramma è il seguente.



Ora facciamo doppio click sulla lifeline di Sistema appena aggiunta e diamole il nome “s” per indicare una possibile istanza del sistema. Si tratta di un metodo alternativo per rinominare le lifeline.



Aprendo la scheda **Model Explorer** sull'estrema sinistra, possiamo notare che il modello è stato aggiornato con i nuovi elementi aggiunti, ad esempio l'SSD InserisciContatto. Si noti inoltre la presenza della classe di supporto chiamata Sistema, creata poco fa per associarla alla lifeline omonima, che sarà utile per contenere le operazioni di sistema richieste per modellare l'SSD, come vedremo tra poco.



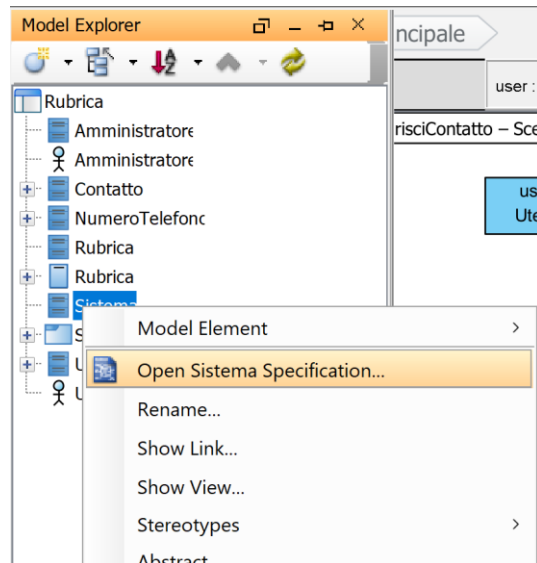
Modellazione scenario principale

Ora che abbiamo inserito le due entità del caso d'uso **InserisciContatto**, ossia l'Utente e il Sistema, vogliamo modellare l'interazione tra di esse attraverso lo scambio di messaggi, che rappresentano **eventi di sistema**, che il sistema dovrà soddisfare mediante l'esecuzione di **operazioni di sistema**.

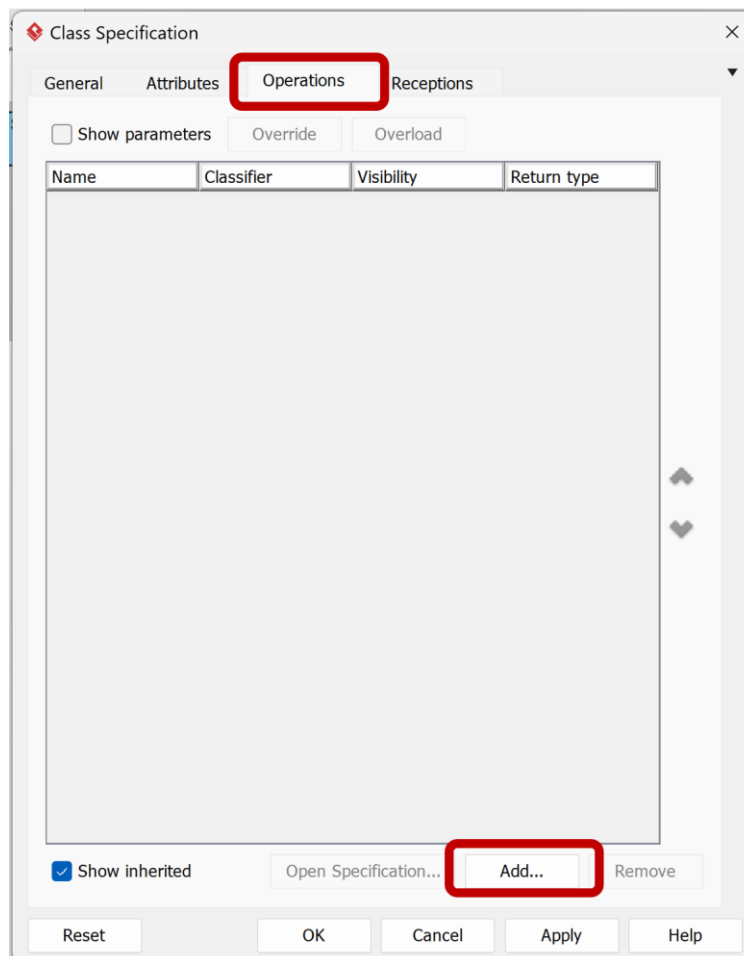
Seguendo la struttura dello scenario principale di successo del caso d'uso, come riportato a inizio documento, un primo messaggio riguarda la richiesta di inserimento di un nuovo contatto, che possiamo chiamare "inserisciNuovoContatto", a partire da Utente verso Sistema.

Per modellare l'evento di sistema, in primo luogo dobbiamo aggiungere l'operazione di sistema inserisciNuovoContatto alla classe Sistema creata poco fa.

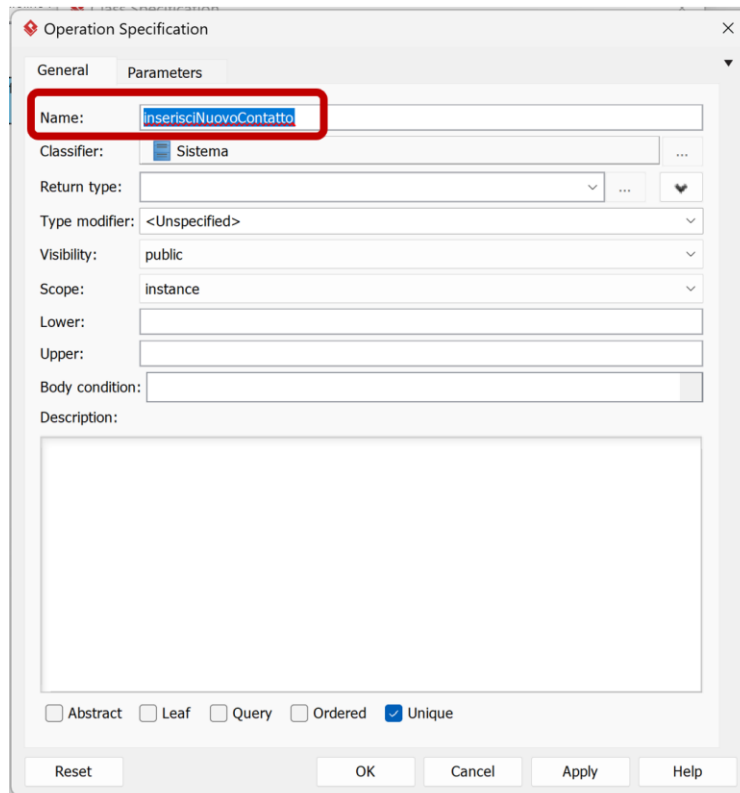
Dal **Model Explorer**, clicchiamo col tasto destro sulla classe Sistema, quindi selezioniamo **Open Sistema Specification**.



Dalla finestra che si apre, clicchiamo sulla scheda **Operations** e poi su **Add**.

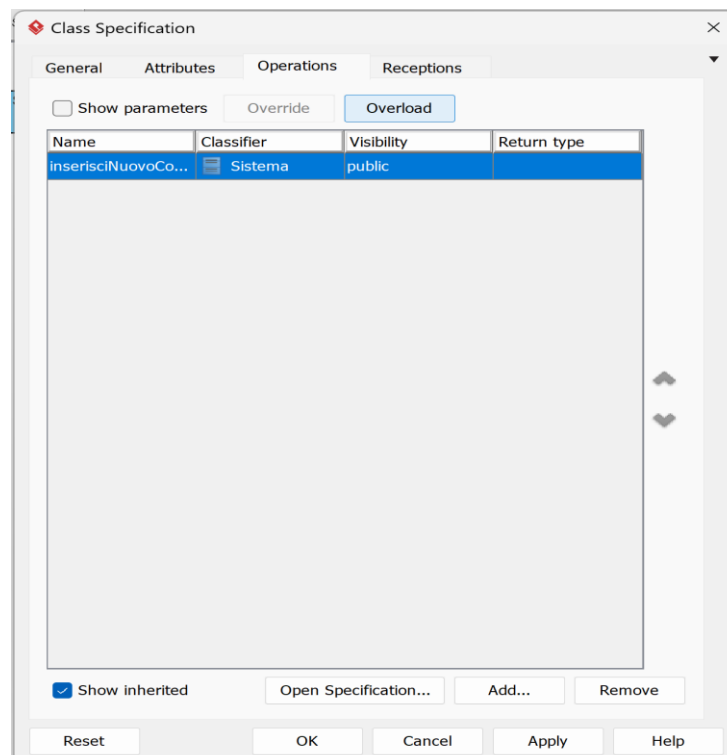


Nella nuova finestra, inseriamo “inserisciNuovoContatto” alla voce **Name** e confermiamo.



The 'Operation Specification' dialog box is shown with the 'Parameters' tab selected. The 'Name' field is highlighted with a red rectangle and contains the text 'inserisciNuovoContatto'. Other fields include 'Classifier' (Sistema), 'Return type' (empty), 'Type modifier' (<Unspecified>), 'Visibility' (public), 'Scope' (instance), 'Lower' (empty), 'Upper' (empty), 'Body condition' (empty), and 'Description' (empty). At the bottom, there are checkboxes for 'Abstract', 'Leaf', 'Query', 'Ordered', and 'Unique' (checked). Buttons at the bottom include 'Reset', 'OK', 'Cancel', 'Apply', and 'Help'.

Il risultato sarà quello di aggiungere l'operazione alla lista di operazioni di Sistema. Clicchiamo **OK**.

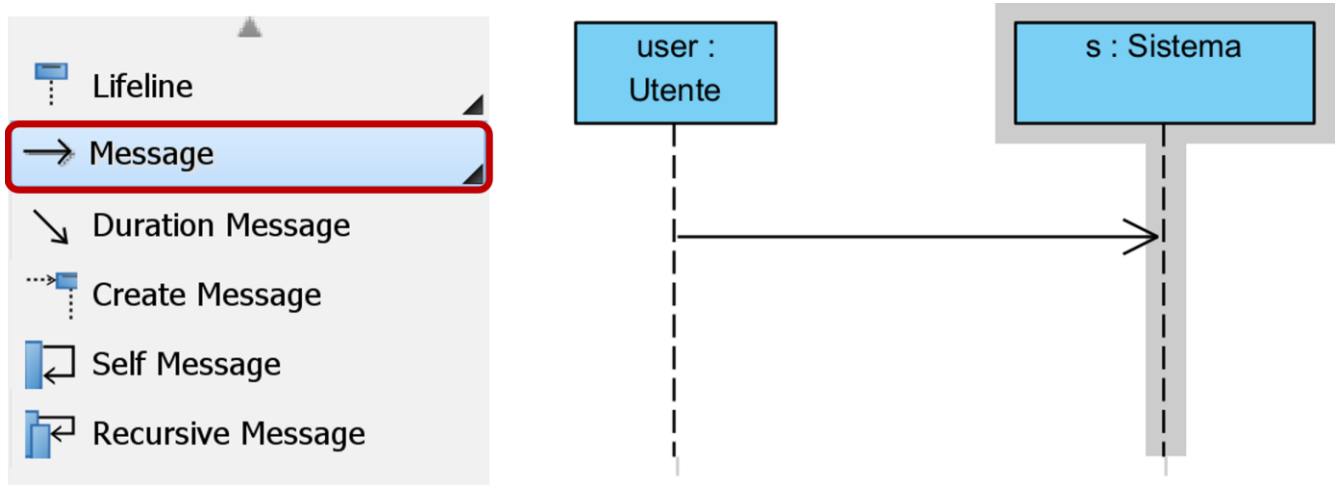


The 'Class Specification' dialog box is shown with the 'Operations' tab selected. The 'Show parameters' checkbox is unchecked. The 'Override' and 'Overload' buttons are visible. A table lists the operations:

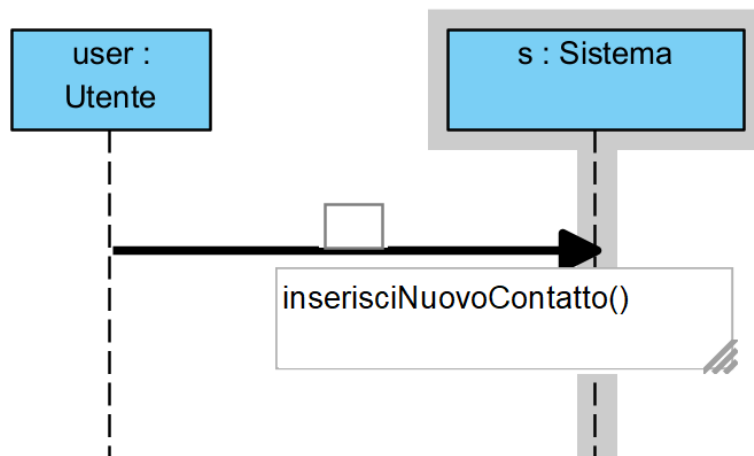
Name	Classifier	Visibility	Return type
inserisciNuovoCo...	Sistema	public	

Below the table, there is a 'Show inherited' checkbox (checked) and buttons for 'Open Specification...', 'Add...', and 'Remove'. At the bottom, there are buttons for 'Reset', 'OK', 'Cancel', 'Apply', and 'Help'.

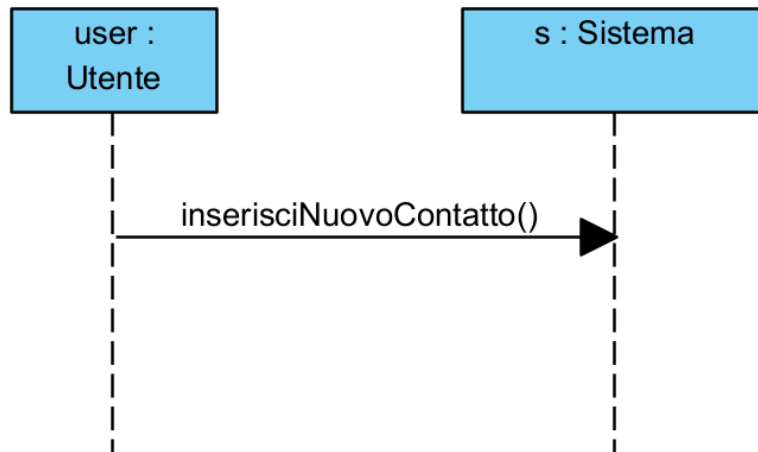
Ora che abbiamo inserito l'operazione di sistema nella classe Sistema, possiamo selezionare dalla **Palette** a sinistra la voce **Message** e trascinare dalla lifeline Utente alla lifeline Sistema.



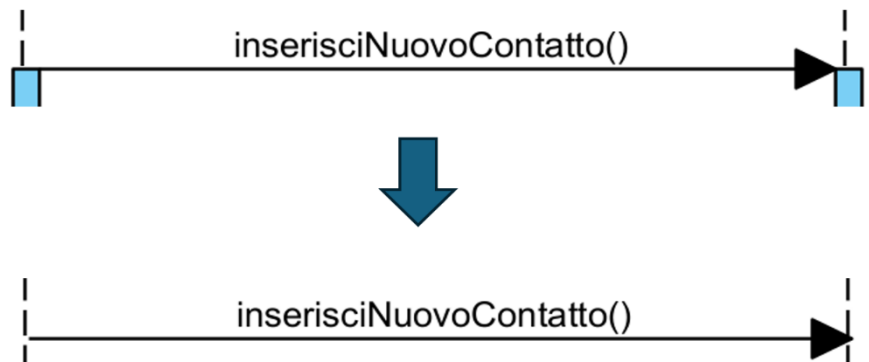
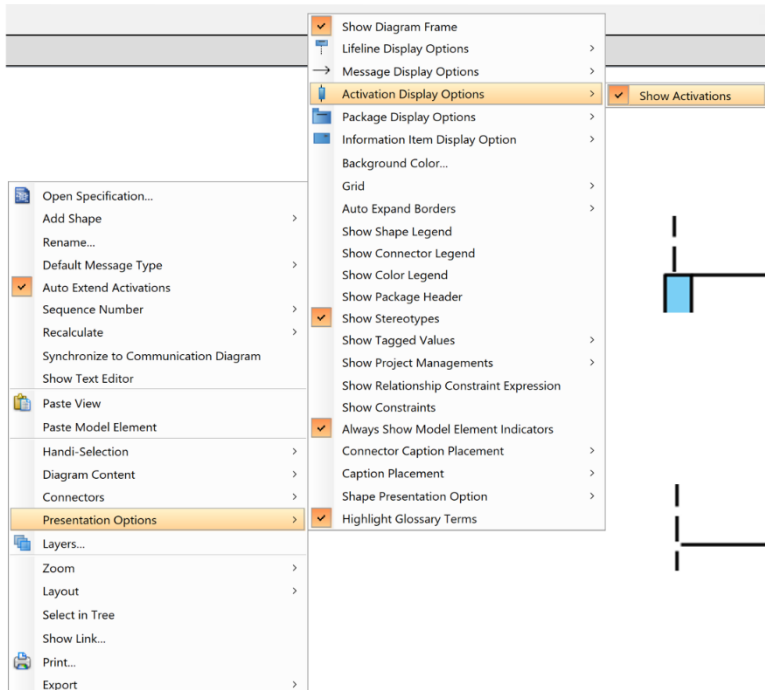
Una volta rilasciato il messaggio (evento di sistema), verrà proposto di associarlo all'operazione di sistema "inserisciNuovoContatto", in quanto presente nella classe Sistema.

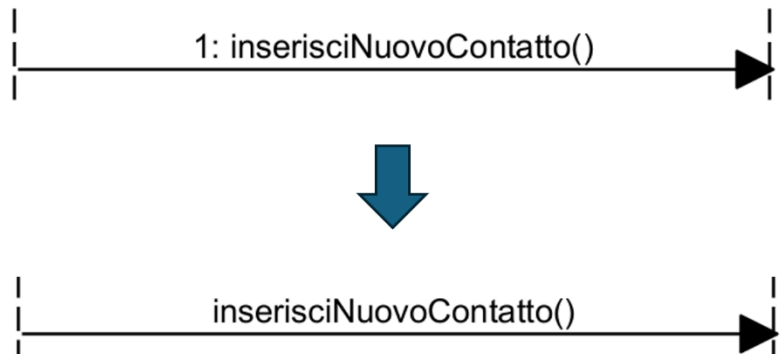
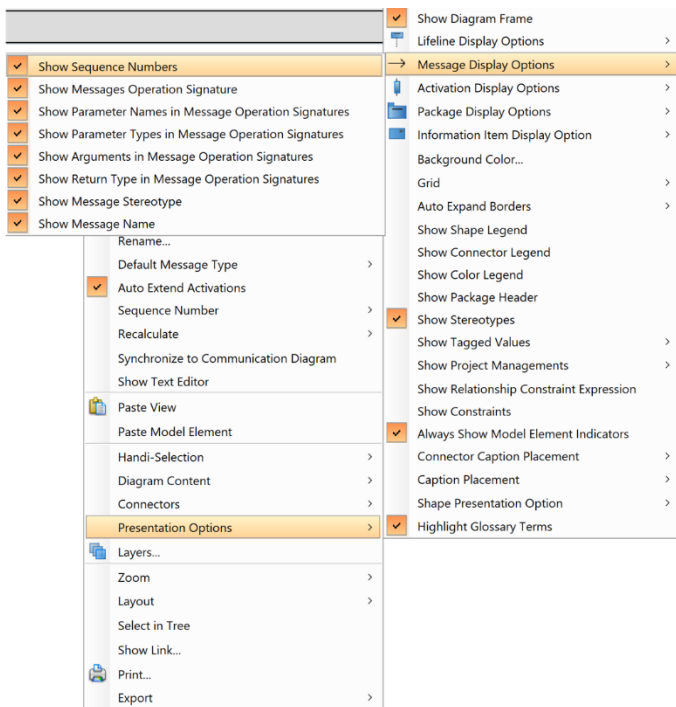


Selezioniamo la voce proposta cliccandoci sopra. Il risultato è il seguente.

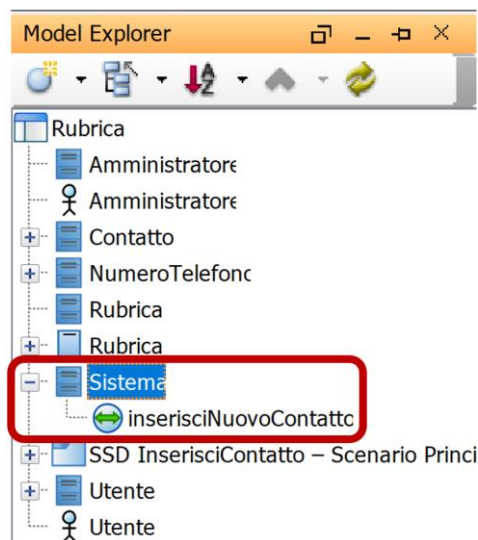


NOTA: In VP è possibile personalizzare i dettagli visualizzati in un SSD, ad esempio si può nascondere il numero sequenziale dei messaggi o le barre di attivazione blu che vengono solitamente create in corrispondenza di un messaggio inserito. Per nascondere questi dettagli (o per visualizzarli nuovamente), si può cliccare con il tasto destro su uno spazio vuoto del diagramma, quindi selezionare **Presentation Options > Activation Display Options > Show Activations** (per visualizzare/nascondere le barre di attivazione) oppure **Presentation Options > Message Display Options > Show Sequence Numbers** (per visualizzare/nascondere i numeri associati ai messaggi).

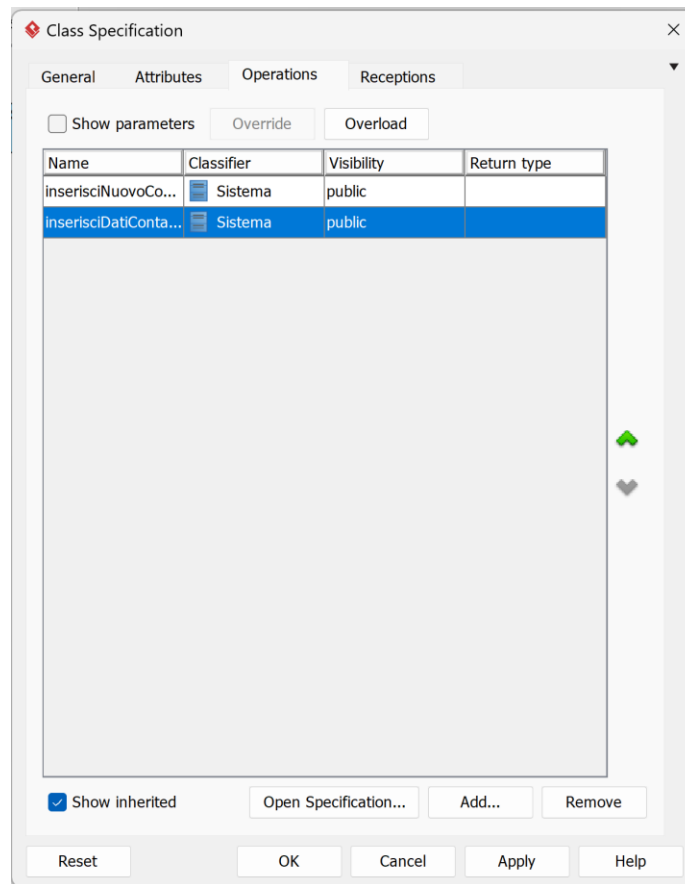




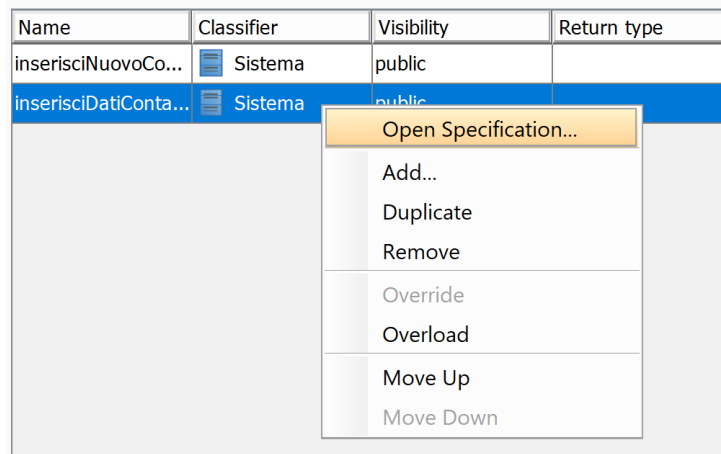
Esaminiamo ora la classe Sistema, sotto la scheda **Model Explorer**. Noteremo come, espandendone la voce, la classe si sia aggiornata e ora include l'operazione di sistema *inserisciNuovoContatto* aggiunta poco fa. Ciò dimostra come il modello tenga traccia di ogni aspetto modellato nei diagrammi, permettendone il riuso.



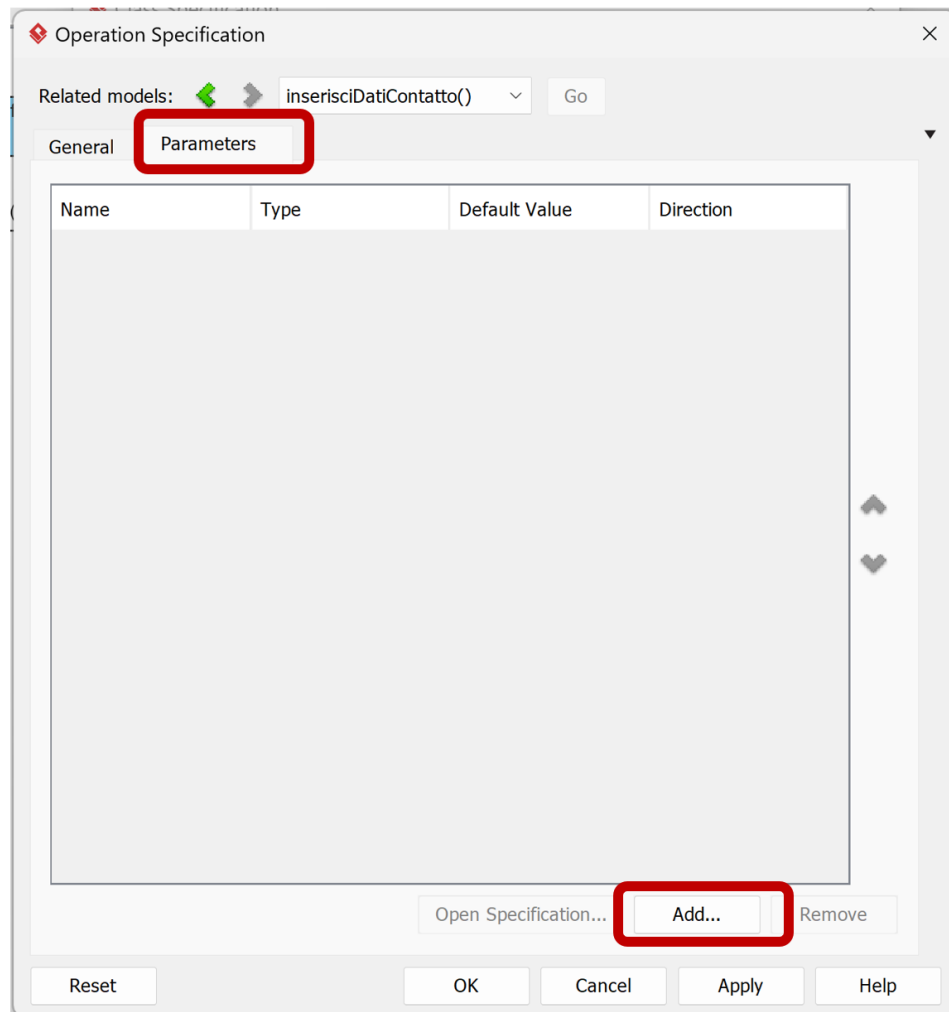
Ora che abbiamo inserito la richiesta di creazione contatto, possiamo modellare attraverso un altro messaggio l'evento di sistema relativo all'inserimento di dati del contatto. Come nel caso precedente, creiamo prima l'operazione di sistema *inserisciDatiContatto*: dal **Model Explorer**, clicchiamo col tasto destro sulla classe Sistema, selezioniamo **Open Sistema Specification** (ci sono molti altri modi per farlo, noi ne vediamo uno solo), quindi **Operations, Add** e aggiungiamo l'operazione.



Diversamente dal messaggio precedente, dobbiamo aggiungere due parametri richiesti dal caso d'uso: "nome" e "descrizione" del contatto. A partire dalla finestra in cui ci troviamo, clicchiamo col tasto destro sull'operazione di sistema *inserisciDatiContatto*, e selezioniamo **Open Specification**.



Dalla finestra che si apre, clicchiamo sulla scheda **Parameters**, quindi clicchiamo il bottone **Add**.



Dalla finestra di inserimento di parametro inseriamo “nome” nel campo **Name**, lasciando il resto inalterato (potremmo specificare anche il tipo di parametro via campo **Type**, es. selezionando “string” dalla lista, ma ai fini della lezione non è necessario) e confermiamo.

Parameter Specification

General

Name: nome

Operation: inserisciDatiContatto

Type: [dropdown]

Type modifier: <Unspecified>

Direction: inout

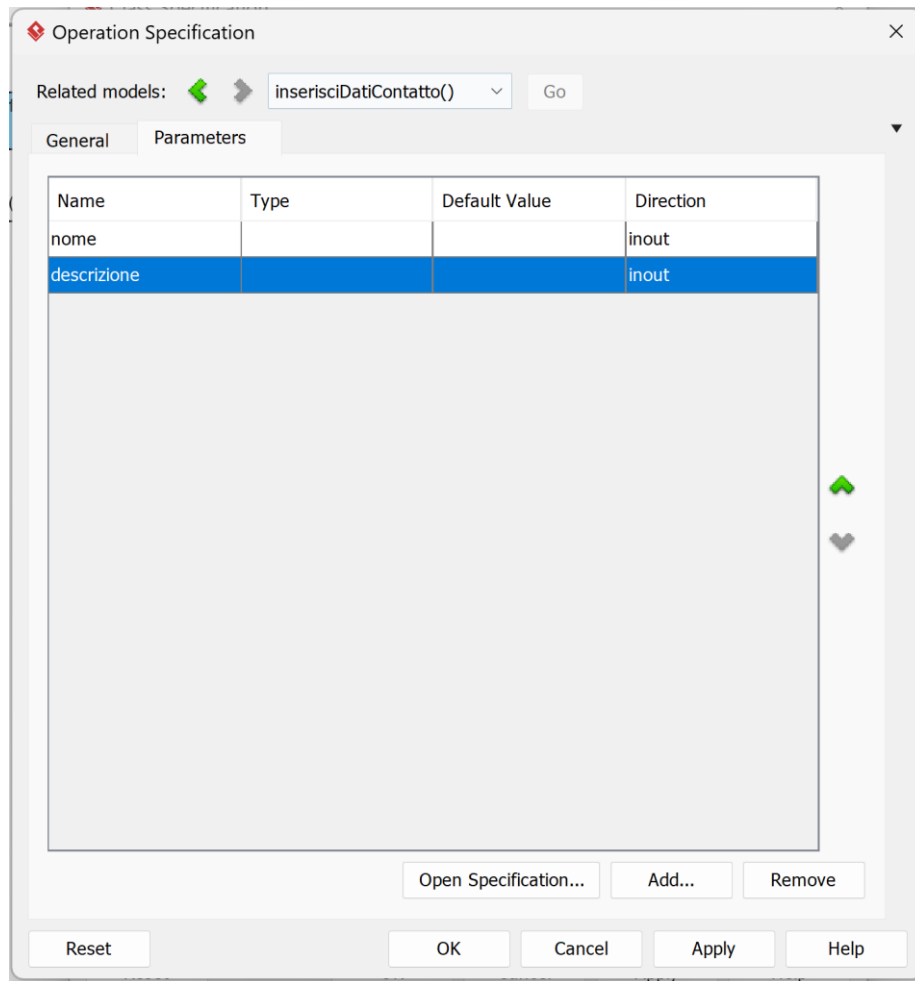
Default value: [dropdown]

Multiplicity: <Unspecified> ☐ Ordered ☒ Unique

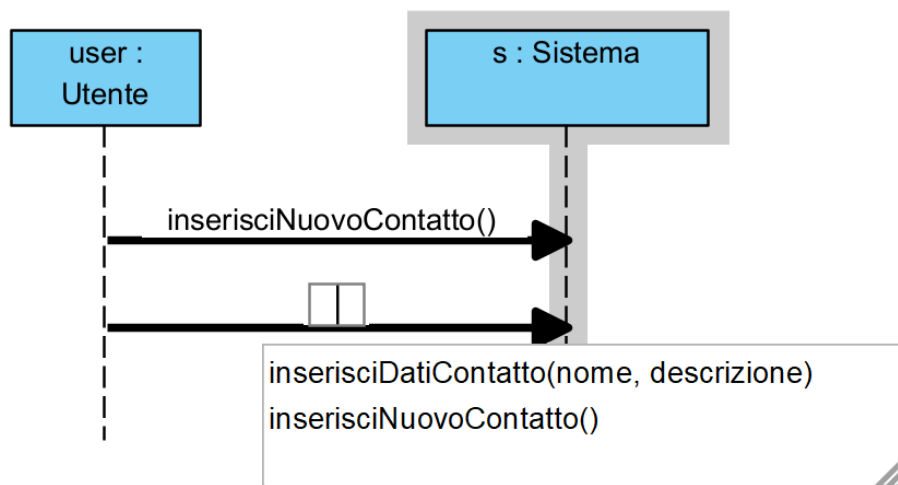
Description:

Reset OK Cancel Apply Help

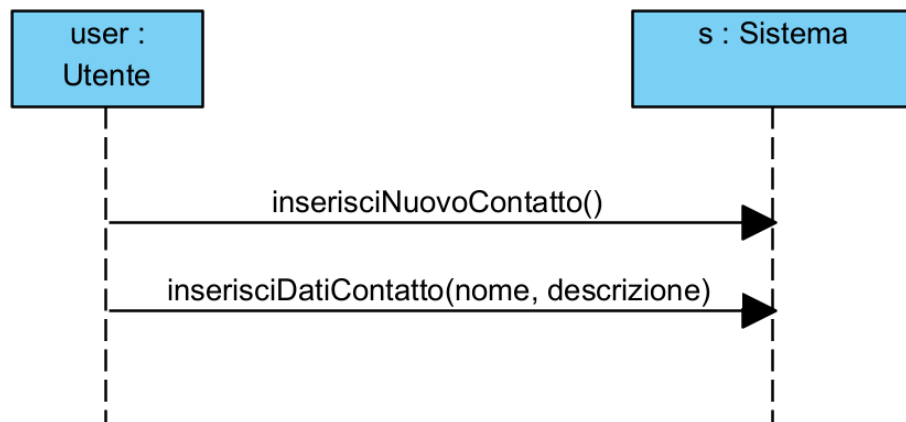
Ripetiamo l'operazione per aggiungere il parametro "descrizione", ottenendo quanto segue.



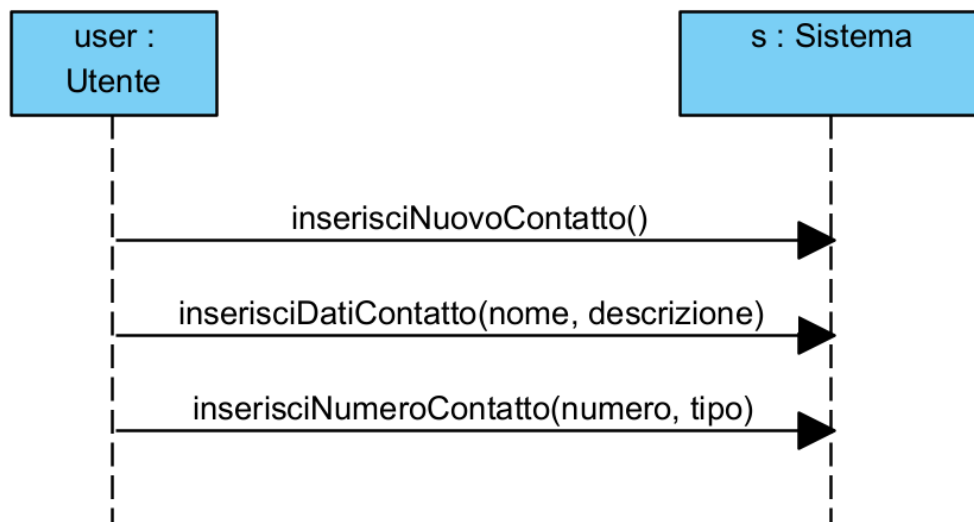
Confermiamo infine l'operazione cliccando su **OK**. A questo punto, possiamo finalmente inserire il messaggio nell'SSD, selezionando la voce **Message** dalla **Palette** e trascinando dalla lifeline Utente alla lifeline Sistema, selezionando la voce `inserisciDatiContatto(nome, descrizione)`.



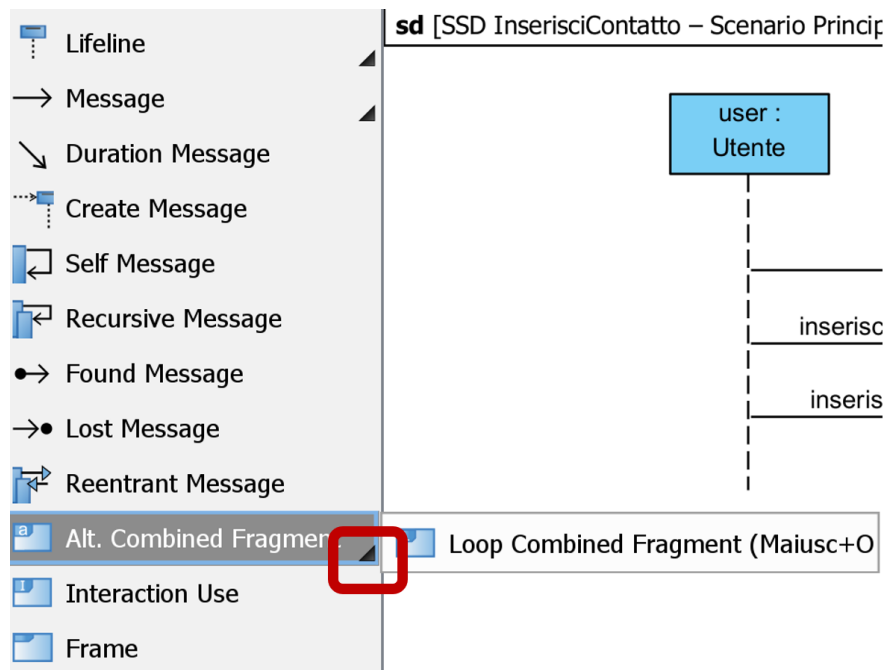
Trascinando le lifeline nel diagramma, come le frecce relative ai messaggi, è possibile riorganizzare il contenuto dell'SSD in modo da renderlo più leggibile.



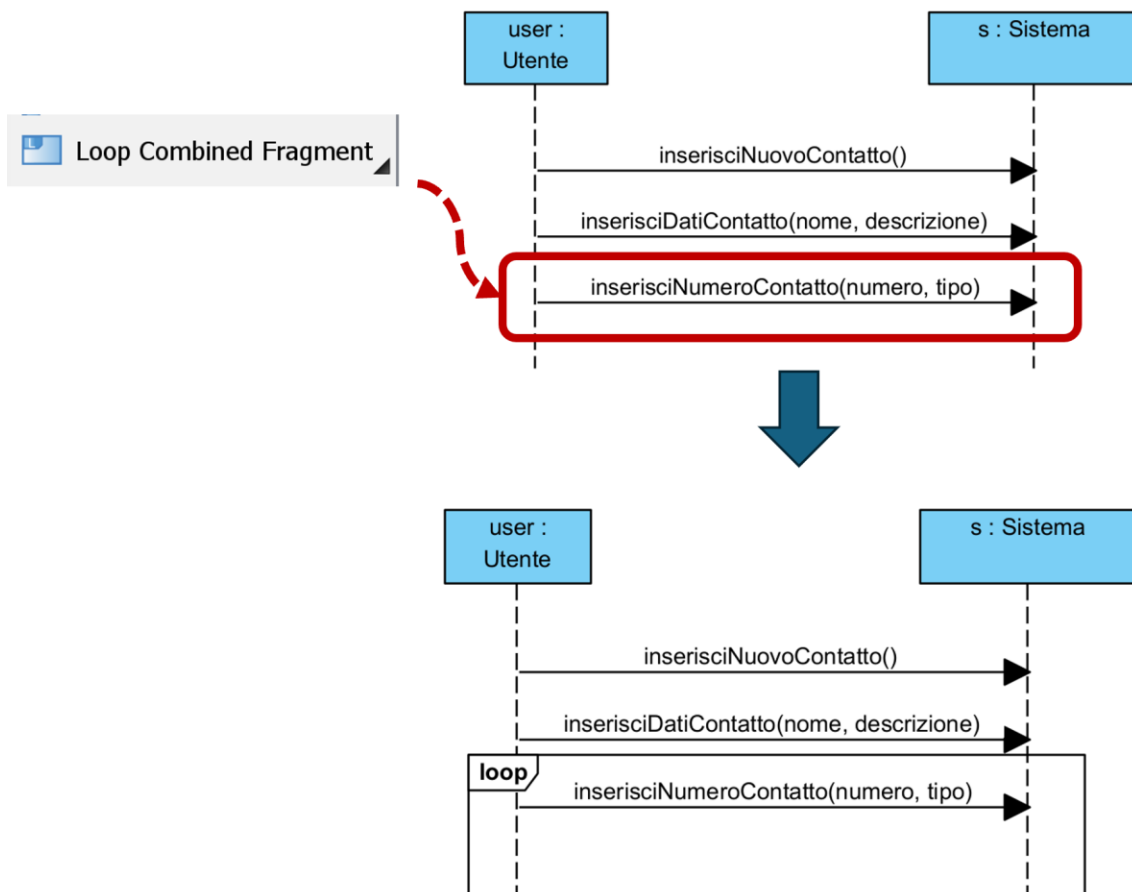
A questo punto dobbiamo modellare l'inserimento di uno o più numeri telefonici del contatto. La ripetizione di questo evento verrà modellata mediante un frame di tipo loop, ossia di iterazione. Come prima cosa creiamo l'operazione di sistema *inserisciNumeroContatto*, come visto in precedenza, considerando i parametri "numero" e "tipo" (es. per indicare se numero fisso, cellulare, etc), quindi tracciamo un messaggio da Utente a Sistema che faccia riferimento a tale operazione di sistema.



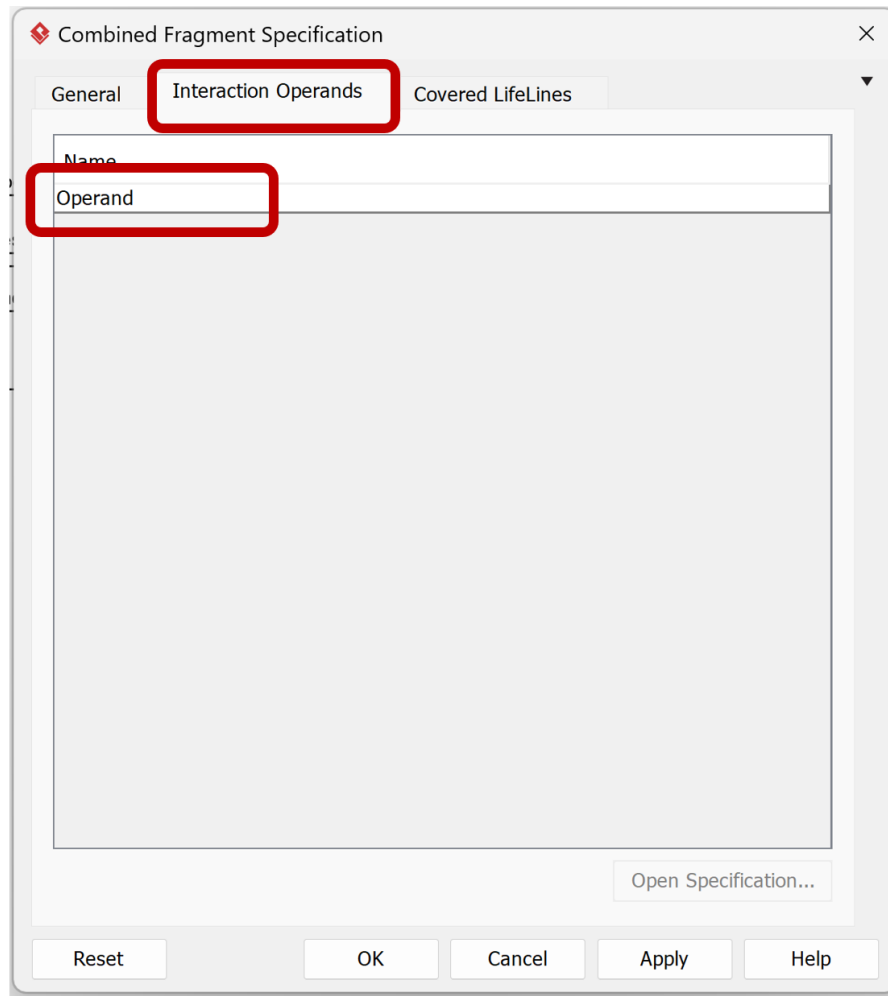
Per aggiungere un frame di tipo loop, dalla **Palette** clicchiamo sulla freccia nera di fianco alla voce **Alt-Combined Fragment**, espandendo così le opzioni, e selezioniamo **Loop Combined Fragment**.



Trasciniamo il loop in modo da coprire l'area contenente la freccia etichettata con "inserisciNumeroContatto", quindi rilasciamo il mouse. Verrà generato un frame di tipo loop a coprire i messaggi selezionati.



Ora che il frame è stato inserito, ci interessa poter esprimere la guardia (condizione) del frame, che dovrà specificare un'iterazione **da 1 a ***, per indicare l'aggiunta di un numero indefinito di numeri di telefono maggiore o uguale a 1. Clicchiamo sul frame, quindi con tasto destro selezioniamo **Open Specification**. Dalla finestra che si apre, clicchiamo sulla scheda **Interaction Operands**, quindi doppio click su **Operand**, che riflette la sezione principale del frame loop.



Dalla finestra che si apre, clicchiamo su **Guard**, quindi ALTERNATIVAMENTE inseriamo "1..*" nel campo **Constraint** oppure "1" e "*" nei campi **Min int** e **Max int**, e infine confermiamo con **OK**. Per quanto quest'ultima opzione sia in questo caso preferibile, è sufficiente utilizzare il campo **Constraint**, dal momento che più genericamente può essere usato per esprimere guardie/condizioni in linguaggio naturale.

Interaction Operand Specification

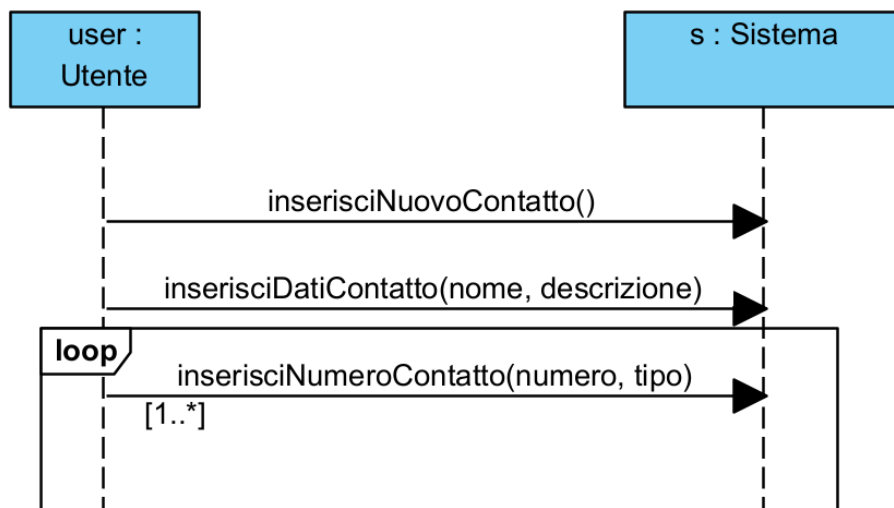
General Guard Messages

Constraint:
1..*

Min int: 1
Max int: *

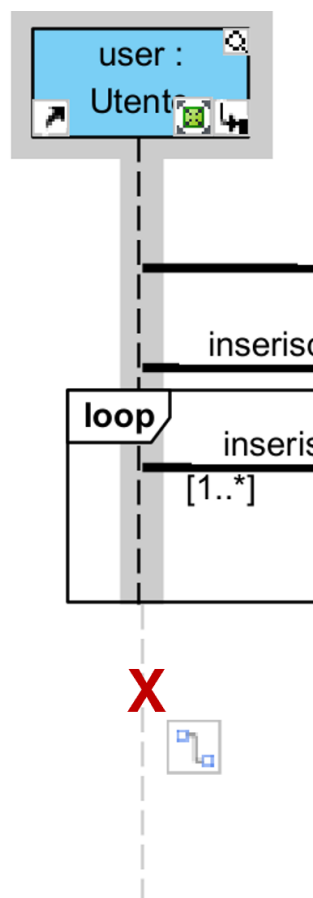
Reset OK Cancel Apply Help

Una volta confermato il tutto, il risultato è il seguente: si noti la guardia del loop tra parentesi quadre.

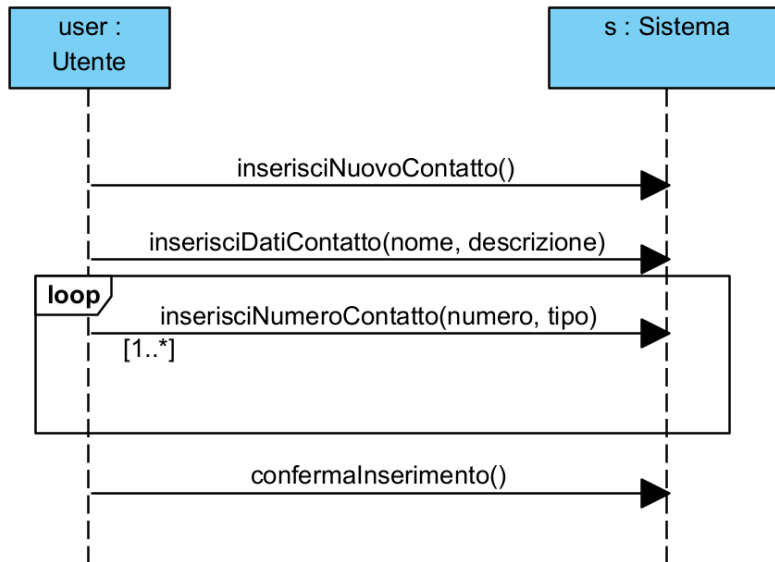


Rimane da aggiungere un ultimo messaggio di conferma di inserimento da parte dell'Utente al Sistema, da posizionare dopo il loop. Creiamo prima l'operazione di sistema *confermaInserimento* nella classe Sistema, come fatto in precedenza (da **Model Explorer**, doppio click sulla classe Sistema, quindi **Open Sistema Specification > Operations > Add**), quindi tracciamo un messaggio via **Palette** dalla lifeline Utente alla lifeline Sistema, da posizionare fuori dal frame di loop in quanto è un evento che verrà invocato una sola volta.

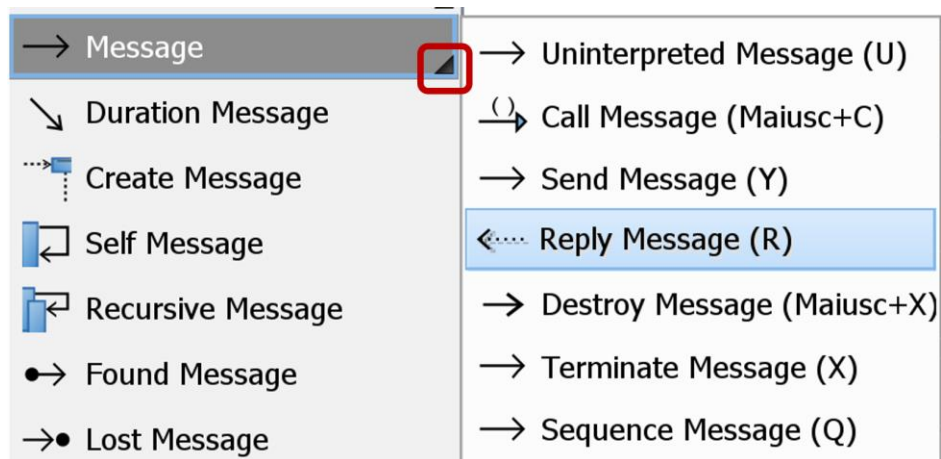
Anche se la lifeline sembra non presentare una linea verticale lunga abbastanza, per tracciare il messaggio fuori dal loop è sufficiente muovere il mouse sopra la lifeline Utente e osservare come esista in realtà una linea verticale tratteggiata più lunga di quella inizialmente mostrata; si può, ad esempio, tracciare il messaggio che ha come partenza il punto indicato dalla **X** della figura seguente.



Il risultato di ciò è il seguente.

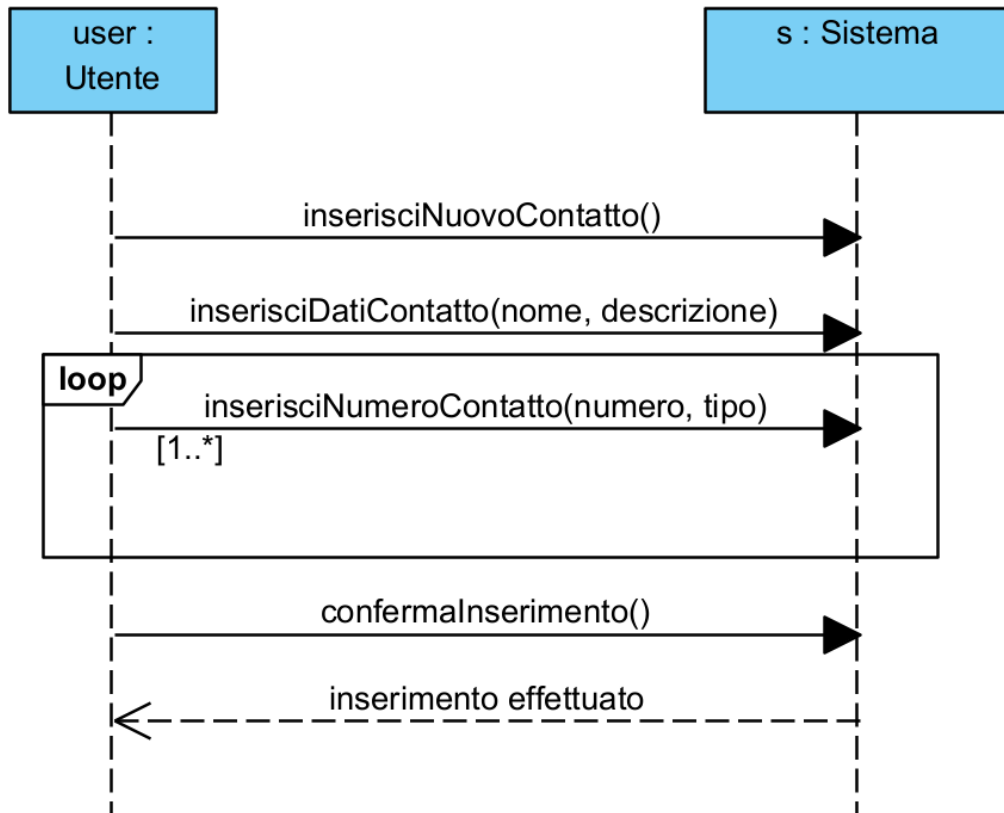


Come ultimo passaggio, in UML è possibile definire messaggi di risposta/valori di ritorno, da parte del Sistema all'Utente, ad esempio per notificare l'inserimento avvenuto con successo del contatto, una volta confermata l'operazione dall'Utente. Per fare questo, selezioniamo la freccetta nera sulla destra della voce **Message** nella **Palette**, e dal menu espanso selezioniamo la voce **Reply Message**.



Tracciamo pertanto un messaggio da Sistema a Utente e, facendo doppio click sul messaggio, inseriamo la stringa "inserimento effettuato"; in questo caso non si tratta di un'operazione di sistema ma di un semplice feedback, quindi è sufficiente manipolare direttamente l'etichetta senza aggiungere operazioni di sistema come fatto in precedenza.

Il risultato finale relativo allo scenario principale del caso d'uso **InserisciContatto** è il seguente.



Contratti

Definiamo ora i contratti per descrivere le operazioni di sistema individuate nell'SSD precedente. Vengono descritte le operazioni di sistema più interessanti, ossia quelle che presentano post-condizioni; escluderemo pertanto l'operazione di sistema `confermaInserimento()` in quanto non presenta post-condizioni significative.

Contratto `inserisciNuovoContatto`

Operazione: `inserisciNuovoContatto()`

Riferimenti caso d'uso: `InserisciContatto`

Precondizioni: è in corso l'aggiunta in Rubrica *r* di un nuovo contatto

Postcondizioni:

- È stata creata un'istanza *c* di `Contatto` (creazione di oggetto)
- Gli attributi di *c* sono stati inizializzati (modifica di attributi)
- l'istanza *c* di `Contatto` è stata associata alla Rubrica *r* (formazione di collegamento)

Contratto `inserisciDatiContatto`

Operazione: `inserisciDatiContatto(nome: string, descrizione: string)`

Riferimenti caso d'uso: `InserisciContatto`

Precondizioni: è in corso l'aggiunta in Rubrica *r* di un `Contatto c`

Postcondizioni:

- *c.nome* è diventato *nome* (modifica di attributo)
- *c.descrizione* è diventato *descrizione* (modifica di attributo)

Contratto `inserisciNumeroContatto`

Operazione: `inserisciNumeroContatto(numero: string, tipo: string)`

Riferimenti caso d'uso: `InserisciContatto`

Precondizioni: è in corso l'aggiunta in Rubrica *r* di un `Contatto c`

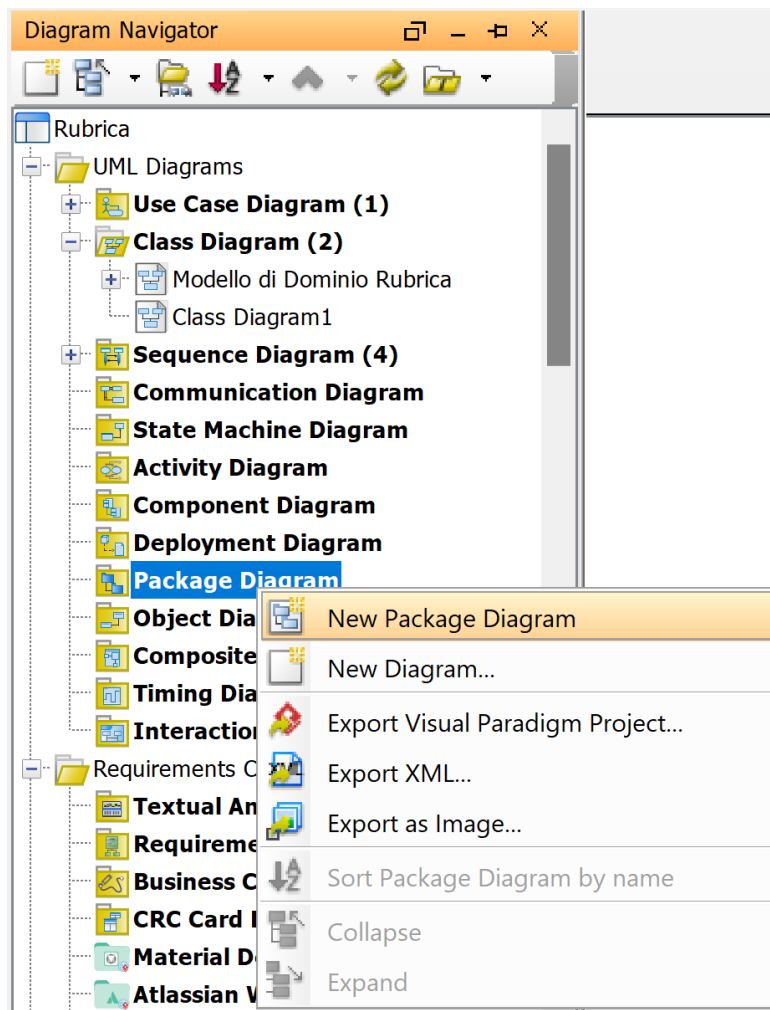
Postcondizioni:

- È stata creata un'istanza *n* di `NumeroTelefono` (creazione di oggetto)
- *n.numero* è diventato *numero* (modifica di attributo)
- *n.tipo* è diventato *tipo* (modifica di attributo)
- l'istanza *n* di `NumeroTelefono` è stata associata al `Contatto c` (formazione di collegamento)

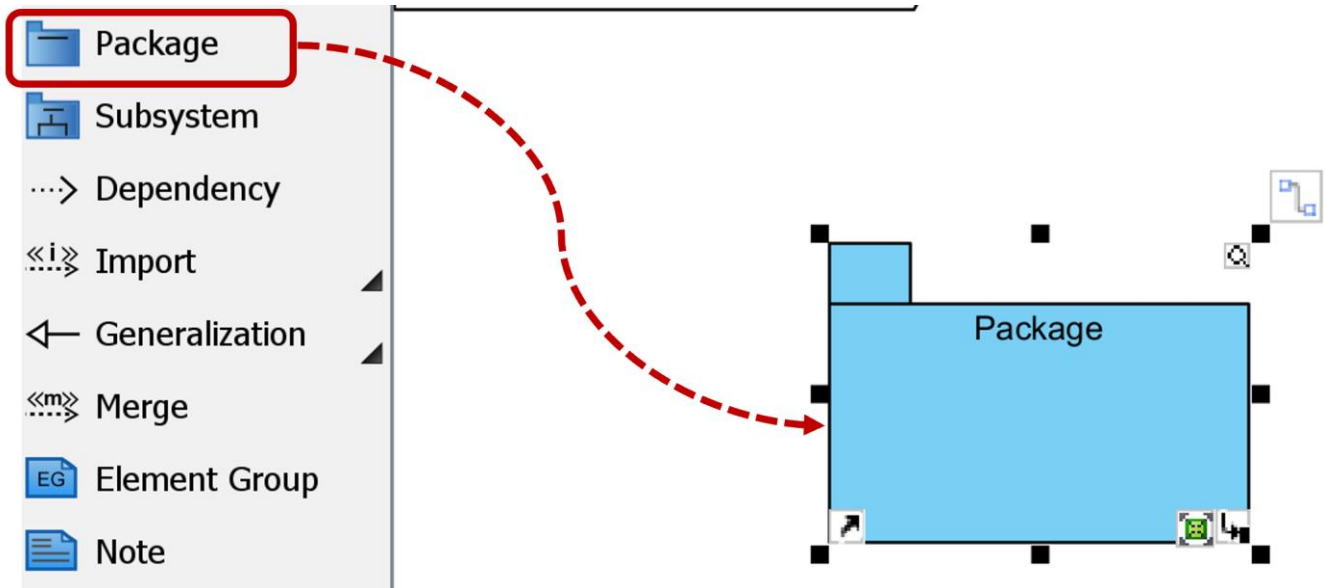
Architettura Logica

Passiamo ora a un preliminare lavoro di progettazione del sistema Rubrica, introducendo l'architettura logica. Assumiamo di voler modellare l'architettura del sistema su quattro strati architetturali: UI, Controller, Dominio, e Servizi Tecnici. Per modellare l'architettura logica del sistema Rubrica in VP ci appoggeremo ai **Package Diagram**.

Come prima cosa, creiamo un diagramma dei package, aprendo la scheda **Diagram Navigator** sulla sinistra e cliccando con il tasto destro sulla voce **Package Diagram**, quindi **New Package Diagram**. Una volta creato il diagramma vuoto, modifichiamo il nome in "Architettura Logica Rubrica".

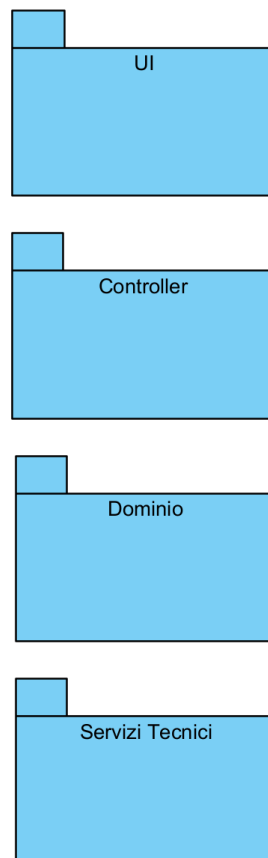


Ora aggiungiamo il primo package che rappresenterà lo strato di UI: selezioniamo **Package** dalla **Palette** a sinistra e trasciniamolo nell'area del diagramma.

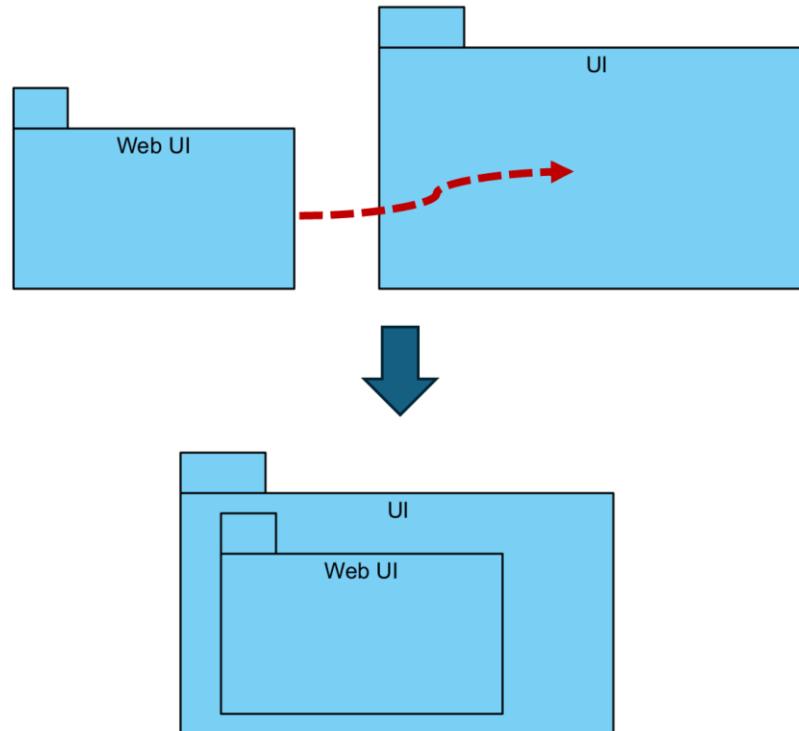


Rinominiamo il package in “UI” via doppio click su di esso. Ripetiamo quanto sopra per i package “Controller”, “Dominio”, e “Servizi Tecnici”, ottenendo quanto segue.

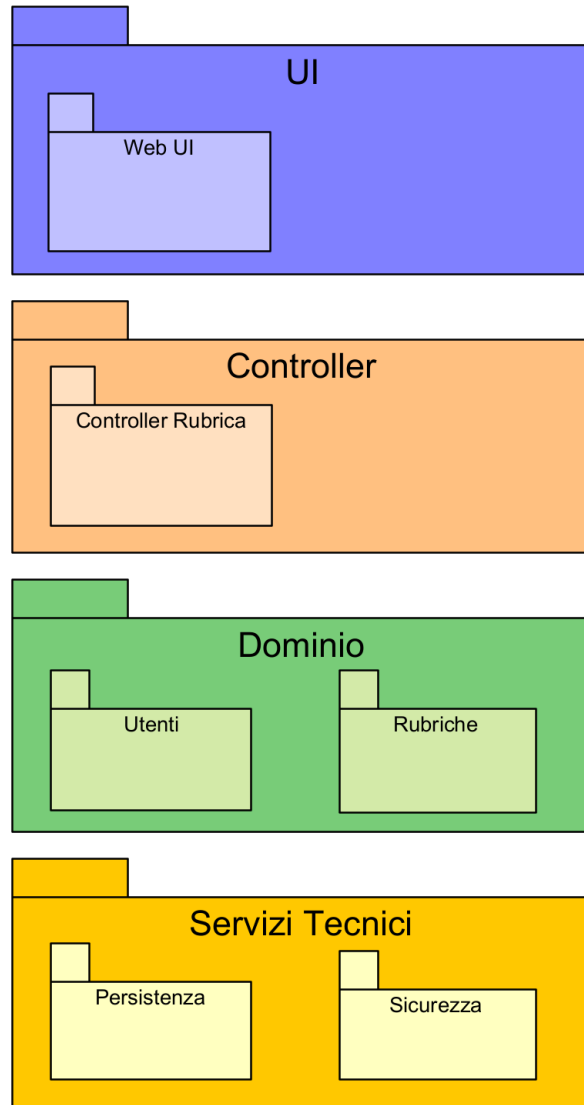
pkg [Architettura Logica Rubrica] /



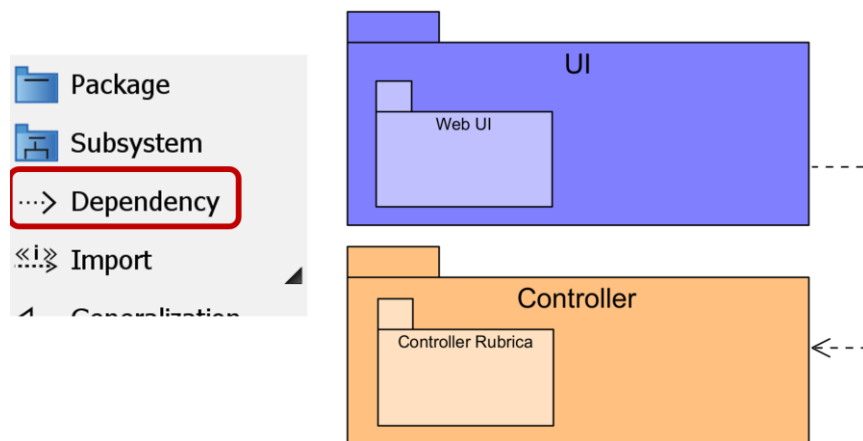
Ora aggiungiamo il package “Web UI” esattamente come sopra. Questo package sarà il contenitore delle classi software che implementeranno l’interfaccia utente. Ingrandiamo il package “UI” trascinandone i bordi e spostiamo il package “Web UI” appena creato all’interno dell’area del package “UI”. Verifichiamo che spostando “UI” venga spostato anche il suo package interno “Web UI”.



Ora ripetiamo quanto sopra per inserire il sotto-package “Controller Rubrica” dentro “Controller” (questo sotto-package si occuperà di smistare gli eventi di interfaccia alle classi dello strato sottostante), i sotto-package “Utenti” e “Rubriche” dentro “Dominio” (questi sotto-package gestiranno le classi software principali relative alle entità del dominio), e infine i sotto-package “Persistenza” e “Sicurezza” dentro “Servizi Tecnici”. Se ritenuto utile, cliccando col tasto destro su un package, quindi **Styles and Formatting > Formats**, è possibile configurarne l’aspetto (es. colore di background, dimensione font).



Rimangono da aggiungere le dipendenze tra packages. Dalla **Palette** a sinistra selezioniamo la freccia **Dependency** e trasciniamola da “UI” a “Controller” (lo strato di interfaccia dipenderà dallo strato sottostante). Adattare la freccia in modo da essere chiaramente visibile.



Ripetiamo la stessa operazione per aggiungere altre dipendenze: una dipendenza da “Controller” a “Dominio”, una dipendenza da “Dominio” a “Servizi Tecnici”, e una dipendenza da “Web UI” a “Sicurezza” (è probabile che alcune classi di interfaccia, durante la sottomissione di form, dovranno affidarsi alle classi sottostanti che gestiscono la sicurezza applicativa). Il risultato ottenuto da quest’operazione è l’architettura logica seguente, base di partenza per la progettazione delle classi software.

